

Phishing Test

Cómo está hecho el analizador de enlaces y el quiz de Marmitako Group

Proyecto: MarmitakoGroup · Vue 3 + Vite · Ruta: `/#/phishing-test` · Rama: `UnaiDev`

1. Qué hace

La página **Phishing Test** tiene dos pestañas:

- **Analizar un enlace:** el usuario pega una URL y la web le dice si parece segura, sospechosa o phishing, enseñándole las señales que ha encontrado y cuál es el dominio real al que lleva.
- **Quiz:** se muestran 8 enlaces al azar (de una lista de 15) y el usuario decide si son legítimos o phishing. Tras cada respuesta se explica el porqué.

Lo importante: **el enlace nunca se abre**. Solo se analiza el texto de la dirección. Opcionalmente se consulta a Google Safe Browsing a través de un servidor propio.

2. Archivos

ARCHIVO	QUÉ CONTIENE
<code>src/utils/analizarUrl.js</code>	El «cerebro»: todas las reglas de detección y la puntuación. JavaScript puro, sin Vue, se puede probar con Node.
<code>src/views/PhishingTest.vue</code>	La página: pestañas, formulario, veredicto y lista de señales. Llama al analizador y a la API.
<code>src/components/UrlColoreada.vue</code>	Pinta la URL por partes para resaltar el dominio real.
<code>src/components/QuizPhishing.vue</code>	El quiz: preguntas, aciertos y resultado final.
<code>server/index.js</code>	Mini servidor Node (sin dependencias) que consulta Google Safe Browsing sin exponer la clave.
<code>src/router/index.js</code> , <code>src/App.vue</code>	Ruta <code>/#/phishing-test</code> y enlace en el menú.
<code>.env.example</code>	Plantilla para la clave de Google (<code>SAFE_BROWSING_KEY</code>).

3. Flujo completo

Usuario pega enlace → pulsa «Analizar»

- |
 - └ 1. analizarUrl(texto) (en el navegador, instantáneo)
 - normaliza → new URL() → aplica reglas → suma puntos → nivel
 - se muestra el resultado al momento
- |
 - └ 2. fetch('api/revisar') (en paralelo, opcional)
 - servidor → Google Safe Browsing → { peligrosa, amenazas }
 - si Google la marca → el veredicto pasa a «Phishing»
 - si no hay servidor o clave → se ignora sin mostrar error

4. El analizador (`analizarUrl.js`)

4.1 Preparar el texto

- Se quitan espacios y se «reactivan» enlaces desactivados como `hxtps://evil[.]com` → `https://evil.com`.
- Si no lleva protocolo (`bit.ly/abc`) se le añade `http://`.
- Si empieza por `javascript:`, `data:`, `vbscript:` o `file:` se marca directamente como peligro (10 puntos).
- Se convierte con `new URL()`, la API estándar del navegador. Si falla, o no tiene ningún punto en el host, se muestra «Enlace no válido».

4.2 Sacar el dominio real

Es la clave de todo: el phishing suele esconder el dominio real detrás de subdominios que parecen legítimos. La función `dominioRegistrado()` se queda con las dos últimas partes del host, o las tres si la penúltima es `co`, `com`, `gob`, `org` ... y la última tiene dos letras:

HOST	DOMINIO REAL
<code>www.bbva.es</code>	<code>bbva.es</code>
<code>bbva.es.acceso-clientes.com</code>	<code>acceso-clientes.com</code> ← el engaño
<code>sede.agenciatributaria.gob.es</code>	<code>agenciatributaria.gob.es</code>
<code>192.168.4.20</code>	<code>192.168.4.20</code> (las IPs se dejan tal cual)

4.3 Reglas y puntos

Cada regla que se cumple añade una señal con un **peso**. Los pesos altos son trucos que casi nunca usa una web legítima.

REGLA	EJEMPLO	PUNTOS
-------	---------	--------

Suplanta una marca conocida (la menciona pero el dominio no es el oficial)	santander.seguridad-clientes.com	5
Imita una marca con letras trucadas	g00gle.com , arnazon.es	5
Imita una marca cambiando una sola letra	amazom.es , pypal	5
Archivo con doble extensión	factura.pdf.exe	5
IP en lugar de dominio (también en hexadecimal)	http://185.23.44.12/	4
Truco de la @ : lo de delante se ignora	paypal.com@evil.tk	4
Letras de otros alfabetos (punycode)	xn--pypal-4ve.com	4
Descarga un ejecutable o comprimido	.exe .apk .zip .msi...	3
«http», «https» o «www» dentro del nombre	https-bbva.net	2
Acortador de enlaces	bit.ly , tinyurl.com	2
Hosting gratuito	web.app , 000webhostapp.com	2
Extensión de riesgo	.xyz .top .tk .zip .click...	2
3 o más subdominios	a.b.c.dominio.com	2
Redirección escondida en los parámetros	?q=https://evil.xyz	2
Protocolo raro (no http/https)	ftp://	2
Palabras gancho (1 → 1 punto, varias → 2). No cuenta en dominios oficiales.	login, verifica, cuenta, multa...	1-2
Sin HTTPS	http://...	1
2 o más guiones en el dominio	correos-entrega-pendiente	1
4 o más números en el dominio	banco83921.com	1
Puerto raro	:8080	1
Enlace de más de 100 caracteres	—	1

4.4 Veredicto

PUNTOS	RESULTADO
0 – 1	Parece seguro
2 – 4	Sospechoso
5 o más (o Google lo tiene en su lista negra)	Phishing

Con estos umbrales, una sola señal grave (5 puntos) ya basta para marcar phishing, mientras que varias leves juntas lo dejan en sospechoso.

4.5 Detección de suplantación de marcas

Hay una lista de ~30 marcas (bancos españoles, Correos, Hacienda, PayPal, Amazon, Netflix, Google, Instagram, Bizum, DGT...) con sus **dominios oficiales**. Si el dominio real está entre los oficiales, no se busca suplantación. Si no, se hacen tres comprobaciones:

1. **Mención directa:** el host contiene el nombre de la marca (`paypal-verifica.xyz`).
2. **Letras trucadas:** se «limpian» el host y la marca cambiando `0→o` , `1→l` , `3→e` , `4→a` , `5→s` , `7→t` , `rn→m` , `vv→w` , `cl→d` , `i→l` , y se vuelve a buscar. Así `g00gle` y `arnazon` se detectan.
3. **Una letra de diferencia:** con la *distancia de Levenshtein* (cuántas letras hay que añadir, quitar o cambiar para pasar de una palabra a otra). Si un trozo del dominio está a distancia 1 de una marca de 5 o más letras, es sospechoso.

```
function distancia(a, b) {
  const d = Array.from({ length: a.length + 1 }, (_, i) => [i])
  for (let j = 1; j <= b.length; j++) d[0][j] = j
  for (let i = 1; i <= a.length; i++)
    for (let j = 1; j <= b.length; j++)
      d[i][j] = Math.min(d[i-1][j] + 1, d[i][j-1] + 1,
        d[i-1][j-1] + (a[i-1] === b[j-1] ? 0 : 1))
  return d[a.length][b.length]
}
// distancia('amazon', 'amazon') === 1 → sospechoso
```

4.6 Lo que devuelve

```
{
  valido: true,
  dominio: 'seguridad-clientes.com',
  puntos: 5,
  nivel: 'peligro', // 'ok' | 'sospechoso' | 'peligro'
  alertas: [{ peso: 5, texto: 'Suplanta a santander (el oficial es
bancosantander.es)' }],
  buenas: ['HTTPS'],
  partes: [ { tipo: 'protocolo', texto: 'https://' }, { tipo: 'subdominio', ... },
    { tipo: 'dominio', ... }, { tipo: 'resto', ... } ]
}
```

5. URL coloreada

El componente `UrlColoreada.vue` recibe `partes` y pinta cada trozo con un estilo distinto, para que el usuario vea adónde lleva el enlace:

```
https://www.paypal.com@paypal-seguro.xyz/
```

- **Subrayado:** el dominio real, lo único que importa.

- Gris: protocolo, subdominios y ruta (los puede inventar cualquiera).
- **Tachado en rojo**: lo que va antes de la `@`, que el navegador ignora.

6. Consulta a Google Safe Browsing

Las reglas solo miran cómo está escrita la URL. Una web de phishing con un dominio normal (`tienda-ofertas.com`) no se detectaría. Para eso se consulta la **lista negra de Google**, la misma que usan Chrome y Firefox.

6.1 Por qué hace falta un servidor

La API necesita una clave. Si se pusiera en el código de Vue, cualquiera podría verla con las herramientas del navegador y gastarla. Por eso `server/index.js` guarda la clave en el archivo `.env` (que no se sube a git) y hace de intermediario:

```
Navegador —POST /api/revisar { url }→ server/index.js —(con clave)→ Google
Navegador ← { peligrosa, amenazas } ← server/index.js ←———— Google
```

En desarrollo, Vite redirige `/api` al puerto 3001 (configurado en `vite.config.js`). En producción, `pnpm start` compila la web y el mismo servidor sirve tanto la web como la API.

6.2 Medidas de seguridad del servidor

- La clave **nunca** sale del servidor.
- Límite de **20 consultas por minuto por IP**, para que nadie agote la cuota.
- Cuerpo de la petición máximo de **4 KB**.
- Solo acepta URLs `http / https` de hasta 2048 caracteres.
- **Timeout de 5 s** a Google: si tarda, se corta.
- Al servir archivos de `dist/` se comprueba que la ruta no se sale de la carpeta (evita *path traversal* tipo `../../etc/passwd`).
- Sin dependencias externas: solo módulos de Node (`node:http` , `fetch`).

6.3 Qué amenazas consulta

TIPO DE GOOGLE	SIGNIFICADO
<code>SOCIAL_ENGINEERING</code>	Phishing o engaño
<code>MALWARE</code>	Distribuye programas maliciosos
<code>UNWANTED_SOFTWARE</code>	Instala software no deseado
<code>POTENTIALLY_HARMFUL_APPLICATION</code>	Apps potencialmente dañinas

Si no hay clave, el servidor responde `503 { configurado: false }` y la página simplemente no muestra nada de Google: el análisis local sigue funcionando.

7. La página (`PhishingTest.vue`)

- Estado con `ref()` : `modo` (pestaña), `enlace` (input con `v-model`), `res` (resultado local) y `google` (respuesta de la API).
 - `computed nivel` : si Google la marca, fuerza «peligro»; si no, usa el nivel local.
 - Un contador `consulta` evita que una respuesta lenta de Google pise el resultado de un enlace analizado después.
 - Las señales se muestran como lista; las graves (peso ≥ 4) en rojo.
 - Accesibilidad: `role="tablist"` y `aria-selected` en las pestañas, `aria-live="polite"` para que los lectores de pantalla anuncien el resultado, y `label` oculto en el input.
-

8. El quiz (`QuizPhishing.vue`)

- Hay 15 preguntas fijas: 7 legítimas y 8 de phishing, cada una con su explicación corta (`porque`).
 - Al empezar se barajan y se cogen 8.
 - Al responder se muestra «Correcto» o «Fallo», la explicación y la URL coloreada (usando el mismo `analizarUrl`).
 - Al final, la nota (`6/8`) y el botón «Otra vez».
 - Se comprobó que el analizador clasifica bien los 15 enlaces del quiz.
-

9. Cómo probarlo

```
pnpm install
pnpm dev                # la web en http://localhost:5173/#/phishing-test

# Opcional: consulta a Google
cp .env.example .env    # y pegar la clave en SAFE_BROWSING_KEY
pnpm api                # en otra terminal
```

Enlaces de prueba:

ENLACE	RESULTADO
<code>https://www.paypal.com/es/signin</code>	Parece seguro (0)
<code>bit.ly/abc</code>	Sospechoso (3)
<code>https://santander.seguridad-clientes.com/acceso</code>	Phishing (5)
<code>https://www.google.com@evil.tk/x</code>	Phishing (6)
<code>http://paypal-verifica-cuenta.xyz/login</code>	Phishing (11)

10. Limitaciones y mejoras posibles

- **Es heurístico:** puede fallar en ambos sentidos. Una web nueva con dominio normal pasa como segura si Google aún no la conoce, y una web legítima con palabras como «login» puede salir sospechosa.
- El cálculo del dominio real es simplificado: no usa la *Public Suffix List* completa.
- La lista de marcas es manual; habría que ampliarla.
- Si se publica solo en GitHub Pages, la consulta a Google no funciona: el servidor necesita un hosting con Node (Render, Railway...).
- Mejoras: consultar también VirusTotal, comprobar la antigüedad del dominio (los de phishing suelen tener días), añadir más preguntas al quiz y guardar récords.